HW 4.3: Reproducible Data Analysis Document

Junbo Zhao

2026-04-12

Table of contents

1 Busiest Airports Analysis **2**
1.1 Data Overview . . . 2
1.2 Visualization . . . 2
1.3 Analysis . . . 2

2 Monte Carlo Numerical Integration **5**
2.1 Method Overview . . . 5
2.2 Small Multiples Visualization . . . 5
2.3 Analysis . . . 5

3 Planning and Prompting GenAI Tools **7**
3.1 Data Context . . . 7
3.2 My Plan for Tidying and Visualizing the Calcium Data . . . 7
3.3 Plan-Informed Visualization . . . 8
3.4 Generic Prompt Results . . . 9
3.5 My Own Visualization . . . 9
3.6 Comparison: Plan-Informed vs. Generic vs. Own Work . . . 9

4 Reflection **13**

Appendix A: GenAI Usage **14**

Appendix B: Code **15**

1 Busiest Airports Analysis

1.1 Data Overview

The table below presents annual passenger traffic (in millions) at six of the world's busiest airports from 2020 to 2025. These airports span four continents and offer a broad view of the global aviation industry: Hartsfield-Jackson Atlanta (ATL), Frankfurt (FRA), Beijing Daxing (PKX), Dubai International (DXB), Los Angeles International (LAX), and London Heathrow (LHR). Data for each airport was sourced from publicly available airport statistics and industry reports.

Table 1: Annual Passenger Traffic (Millions) at Six Major Airports, 2020–2025

Airport	Name	2020	2021	2022	2023	2024	2025
ATL	Hartsfield-Jackson Atlanta	42.9	75.7	93.7	104.7	108.1	106.3
FRA	Frankfurt Airport	18.8	24.8	48.9	59.4	61.6	63.2
PKX	Beijing Daxing	16.0	25.7	17.5	39.4	52.0	53.6
DXB	Dubai International	25.9	29.1	66.1	86.9	92.3	95.2
LAX	Los Angeles International	28.8	41.9	65.9	75.0	76.6	73.7
LHR	London Heathrow	22.1	19.4	61.6	79.2	83.9	84.5

1.2 Visualization

1.3 Analysis

Taken together, Table 1 and Figure 1 tell a compelling story about the pandemic's impact on global air travel and the varied paths airports have taken to recover from it. In 2020, every single airport on this list experienced a dramatic collapse in passenger traffic. Atlanta dropped from a typical volume of over 100 million to just 42.9 million; Heathrow fell to 22.1 million; and Frankfurt bottomed out at 18.8 million. The chart makes this crash visually unmistakable—all six lines plunge downward in 2020.

What stands out from that point onward is just how different the recovery trajectories have been. Dubai International (DXB) has the most striking growth arc: from 29.1 million in 2021, it surged to 86.9 million by 2023 and reached a record 95.2 million in 2025. That pace of recovery is really impressive and reflects Dubai's growing importance as a global transit hub. Atlanta (ATL) also recovered quickly, crossing the 100-million mark again by 2023 and peaking at 108.1 million in 2024. But interestingly, ATL actually dipped slightly to 106.3 million in 2025—suggesting its domestic-heavy traffic base may be reaching a ceiling.

Similarly, LAX showed a notable dip from 76.6 million in 2024 to 73.7 million in 2025, making it the only other airport to decline in the most recent year. This could reflect shifting travel

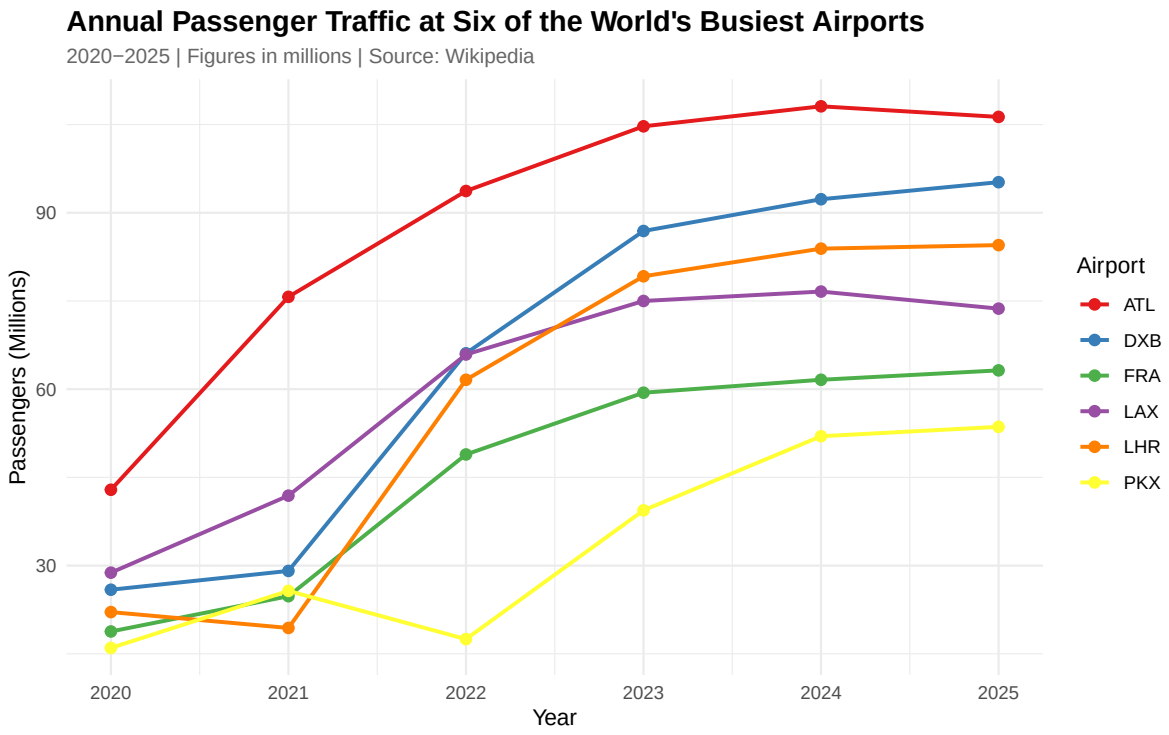


Figure 1: Annual passenger traffic at six of the world's busiest airports, 2020–2025. Each line represents one airport; figures in millions of passengers. Source: airport statistics and industry reports.

patterns, airline capacity changes, or competition from other U.S. hubs. In contrast, Heathrow (LHR) continued its upward trend, setting a new all-time record of 84.5 million in 2025.

Beijing Daxing (PKX) is probably the most interesting outlier in the dataset. While every other airport was climbing upward through 2021 and 2022, PKX actually dropped from 25.7 million to 17.5 million in 2022—a direct reflection of China's strict zero-COVID policies. The table confirms this peculiar dip, and the chart makes it visually obvious since PKX's line dives downward while everyone else is going up. Since then, PKX has recovered quickly, reaching 53.6 million in 2025, though it still lags behind the others in absolute terms.

Frankfurt (FRA), meanwhile, has followed a steady but modest recovery. Its passenger volume climbed to 63.2 million in 2025, which is still about 10% below its pre-pandemic levels. Compared to DXB or ATL, Frankfurt's recovery feels structurally incomplete rather than just delayed.

2 Monte Carlo Numerical Integration

2.1 Method Overview

Monte Carlo integration is a statistical technique for estimating definite integrals using random sampling. The idea is straightforward: generate random points inside a rectangular region that bounds the function, check how many fall on or below the curve, and use that proportion to estimate the area under the curve. Multiply the proportion by the rectangle's area, and you get an estimate of the integral.

For this analysis, I'm integrating the Chi-squared probability density function with df = 5, over the interval $0 \leq x \leq 20$ with $y \in [0, 0.16]$. The bounding rectangle has area $20 \times 0.16 = 3.2$. I wrote a helper function `generate_mcPoints()` that creates the random sample, and a plotting function that visualizes the result.

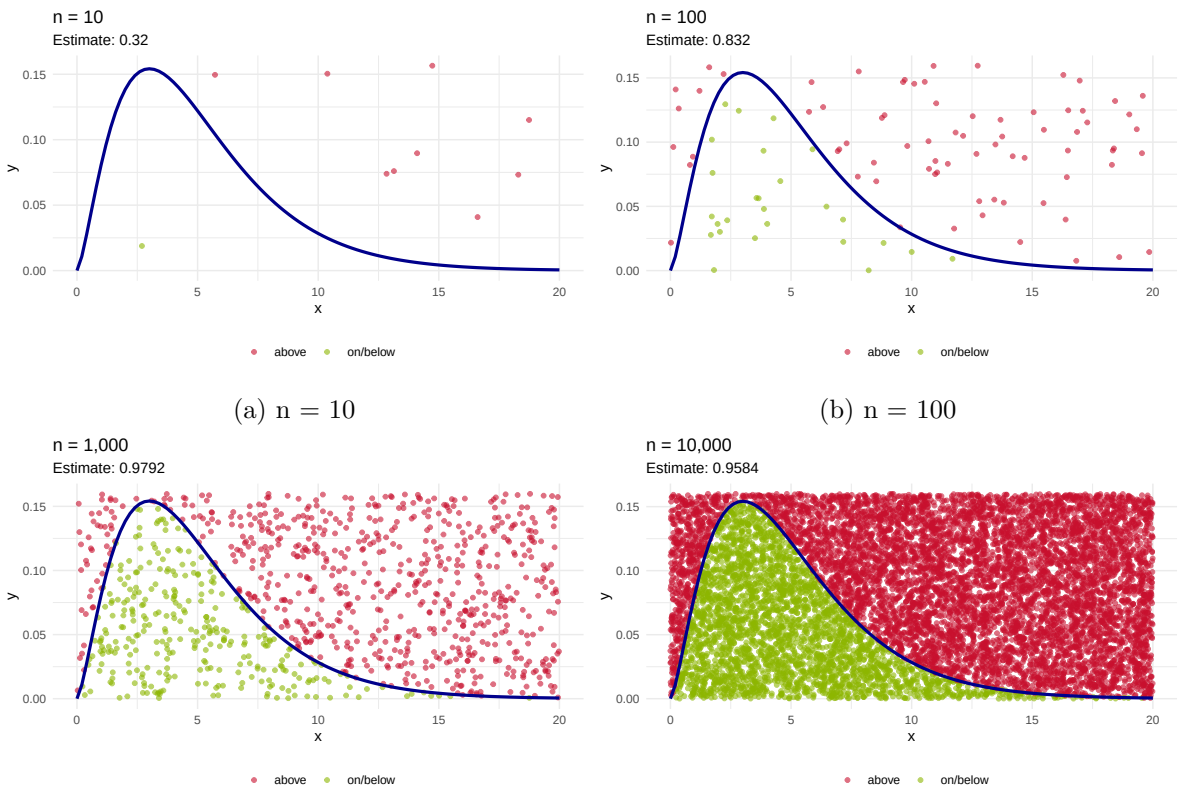
2.2 Small Multiples Visualization

2.3 Analysis

Figure 2 shows four Monte Carlo simulations with sample sizes of 10, 100, 1,000, and 10,000. In each panel, the dark blue curve traces the Chi-squared PDF (df = 5), while the scattered points are colored green (on or below the curve) or red (above it). The estimated integral appears in each panel's subtitle.

With only n = 10 points (Figure 2a), the estimate is highly unreliable—there just aren't enough points to approximate the shape of the density curve in any meaningful way, and the result can swing wildly depending on the random draw. At n = 100 (Figure 2b), you can start to see the general shape of the distribution emerge, though the estimate is still fairly noisy. The jump to n = 1,000 (Figure 2c) is where things really start to come together: the boundary between green and red points begins to trace out the curve quite clearly, and the estimate gets noticeably closer to the true value. By n = 10,000 (Figure 2d), the green region almost perfectly fills in the area under the curve, and the estimate converges very close to 1.0.

This convergence toward 1.0 makes sense from a theoretical standpoint. The Chi-squared distribution with df = 5 is a proper probability density function, meaning its total area integrates to exactly 1. Since our integration domain of $[0, 20]$ captures essentially all of the density (the Chi-squared PDF is negligible beyond x = 20 for df = 5), the Monte Carlo estimate should approach 1.0 as the sample size increases. The small multiples format makes this convergence process visually obvious as you scan from the top-left panel to the bottom-right.



(a) n = 10

(b) n = 100

(c) n = 1,000

(d) n = 10,000

Figure 2: Monte Carlo integration of the Chi-squared PDF (df = 5) at four sample sizes. Green points fall on or below the density curve; red points fall above it.

3 Planning and Prompting GenAI Tools

3.1 Data Context

The calcium dataset comes from a longitudinal study tracking 31 women over four years, measuring calcium levels in the ulna of each woman's dominant arm via photon absorptiometry. Fifteen women served as the null treatment group (no intervention), and the remaining 16 participated in a diet and exercise program. Measurements were taken annually: Initial (y0), Year 1 (y1), Year 2 (y2), and Year 3 (y3).

The CSV file is not tidy—it has 8 columns with duplicated names (y0, y1, y2, y3 appear twice), where columns 1–4 belong to the null group and columns 5–8 belong to the diet/exercise group. There are no group labels or subject identifiers.

3.2 My Plan for Tidying and Visualizing the Calcium Data

Before prompting Claude, I put together a step-by-step plan for how to approach this data:

Goal: Transform the untidy calcium CSV into tidy format and produce a visualization comparing calcium trajectories across the two treatment groups over time.

Identifying what needs fixing:

The column names are duplicated so you can't tell groups apart by name. The two groups sit side by side instead of being stacked. Each row mixes data from two different women. There's no subject ID or group label anywhere in the file.

Steps:

1. Read the CSV using `read_csv()`, which auto-renames duplicate columns, keeping all 8 columns intact.
2. Split by column position: columns 1–4 = null group, columns 5–8 = diet/exercise group.
3. Add a subject ID (N1, N2, … for null; E1, E2, … for exercise) and a group label to each subset.
4. Rename the time columns to Initial, Year1, Year2, Year3.
5. Pivot each group from wide to long format using `pivot_longer()`.
6. Stack the two groups with `bind_rows()`.
7. Drop NA rows (the null group has only 15 subjects, so row 16 is all NA).
8. Create a group-level summary plot showing mean calcium over time with standard error bars.

Checkpoints: After step 6, confirm the tidy data frame has the expected number of rows (31 subjects * 4 time points = 124 rows, minus 4 NAs = 120). After step 8, verify the plot shows diverging or converging trends between groups.

3.3 Plan-Informed Visualization

The following R code and visualization were generated by Claude (claude-opus-4-6, April 12, 2026) using the plan above as context, along with the calcium.csv file.

Prompt used:

> I have a calcium.csv file from a longitudinal study of 31 women over 4 years measuring ulnar calcium levels. The data has 8 columns with duplicate names (y0,y1,y2,y3 repeated twice). The first 4 columns are the null treatment group (15 women, no intervention) and the last 4 are the diet/exercise group (16 women). Here is my plan: (1) Read the CSV preserving all 8 columns despite duplicate names. (2) Separate into two groups by column position. (3) Add subject IDs and group labels. (4) Rename time columns to Initial, Year1, Year2, Year3. (5) Pivot each group to long format. (6) Combine into one tidy data frame. (7) Remove NAs. (8) Create a line plot comparing mean calcium over time by group with standard error bars. Please write R code using tidyverse to execute this plan.

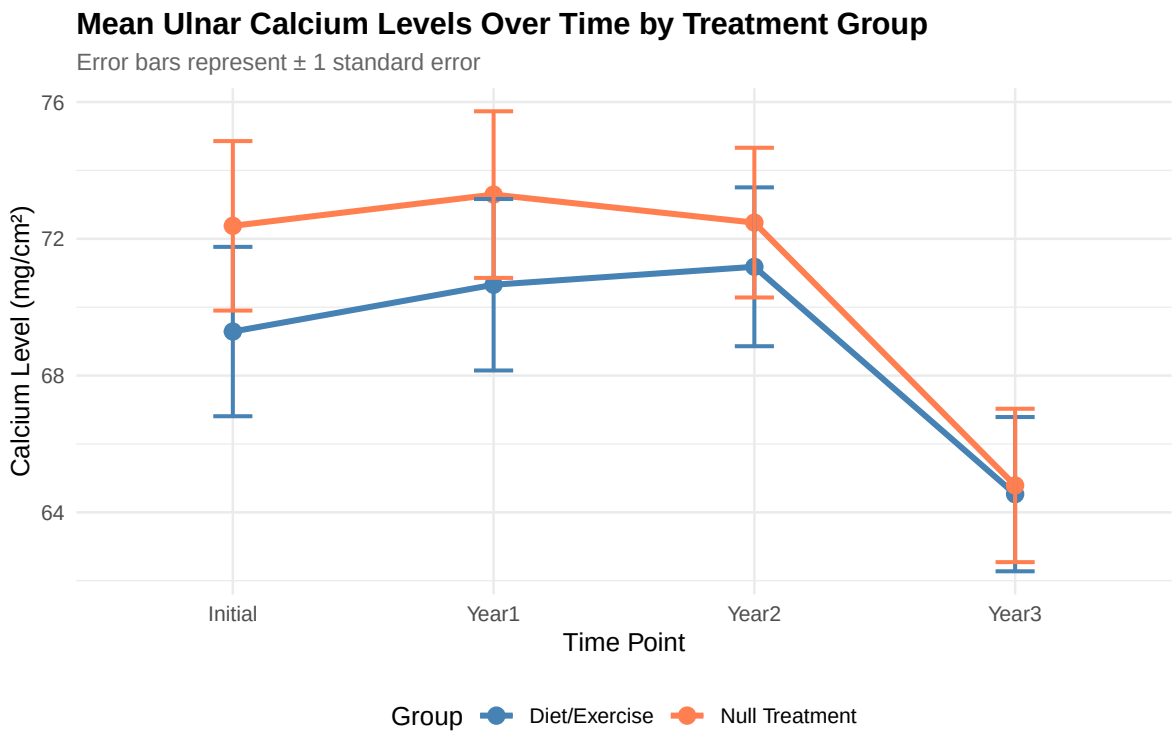


Figure 3: Plan-informed visualization: Mean ulnar calcium levels over time by treatment group, with standard error bars. Generated by Claude using a detailed step-by-step plan.

3.4 Generic Prompt Results

The following two figures are the output from the instructor-provided generic prompt given to Claude Sonnet 4.6 Extended (March 31, 2026). The generic prompt simply asked Claude to analyze the calcium data without any specific plan or guidance about the data structure.

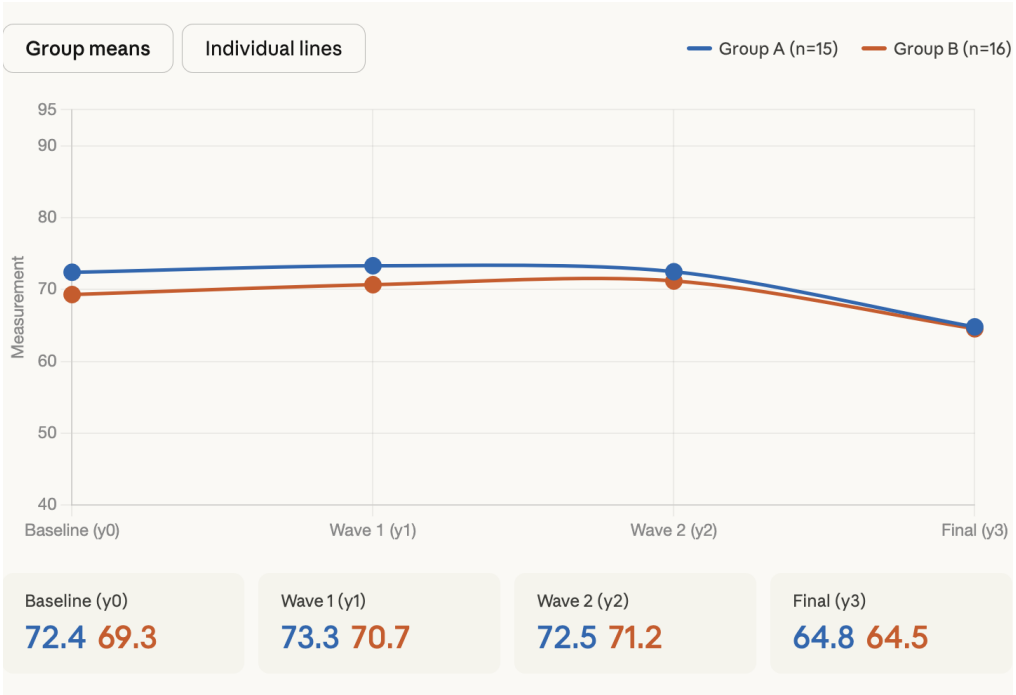


Figure 4: Generic prompt output: Group means over time. Produced by Claude Sonnet 4.6 Extended using an unguided prompt.

3.5 My Own Visualization

To add a third perspective beyond the plan-informed and generic outputs, I created a faceted individual-trajectory plot that lets you see every woman's calcium path within each group side by side.

3.6 Comparison: Plan-Informed vs. Generic vs. Own Work

Comparing these three sets of outputs reveals some clear differences in how the level of prompt specificity shapes the quality and usefulness of AI-generated results.

The plan-informed output (Figure 3) produced exactly what I asked for: a clean group-means plot with standard error bars, proper axis labels, and a color scheme that makes the two groups

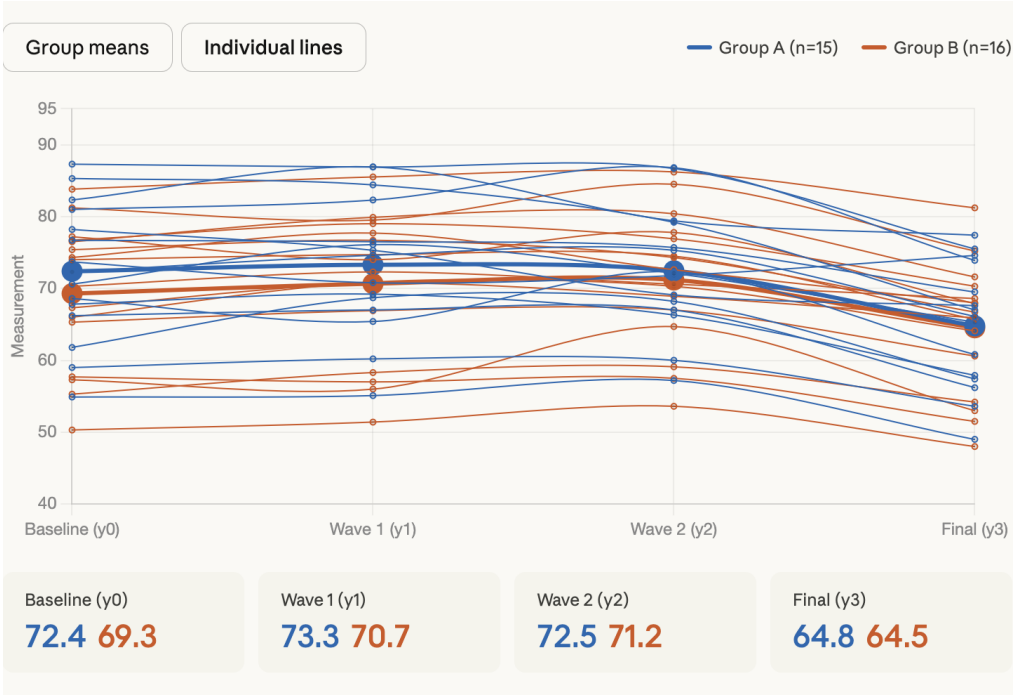


Figure 5: Generic prompt output: Individual trajectories. Produced by Claude Sonnet 4.6 Extended using an unguided prompt.

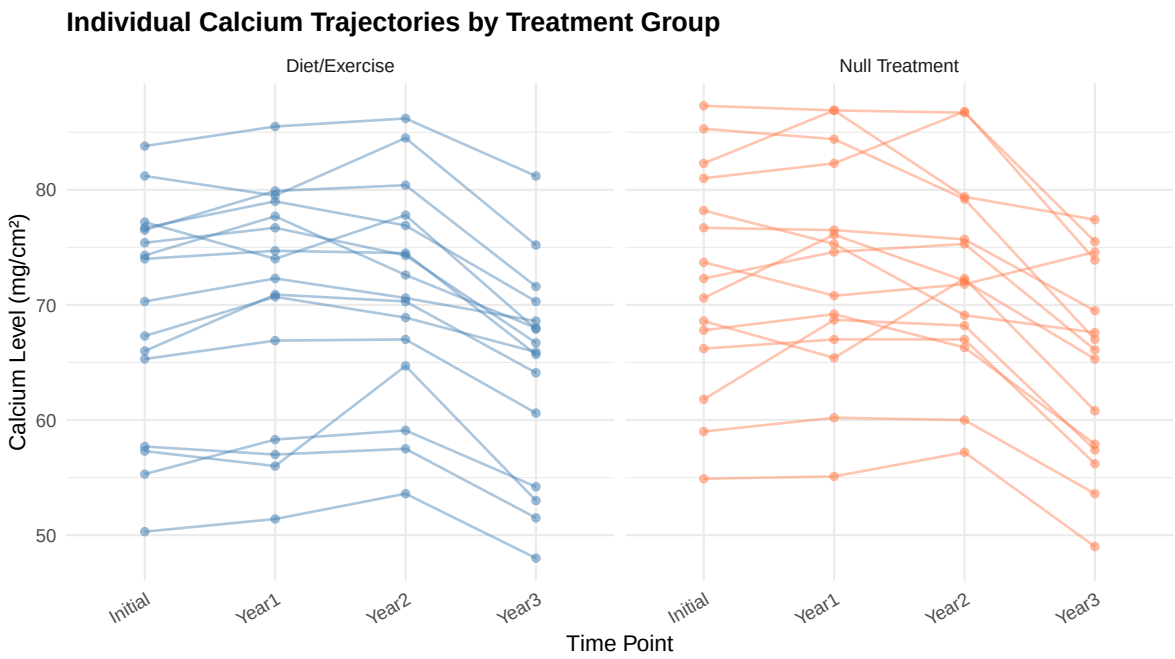


Figure 6: Individual calcium trajectories faceted by treatment group. Each line represents one woman's calcium measurements over the four time points.

easy to distinguish. Because the prompt explicitly described the data structure (duplicate column names, group positions, 15 vs. 16 subjects), the generated code handled the tricky tidying steps correctly on the first try. The error bars are especially useful because they give you a sense of the uncertainty around the group means.

The generic output (Figure 4 and Figure 5) managed to produce reasonable-looking charts, but there are clear limitations. The group means plot uses generic labels like "Group A" and "Group B" rather than the actual treatment names, which makes interpretation harder. It also lacks error bars, so you can't judge whether the apparent differences between groups are meaningful or just noise. The individual trajectories plot (Figure 5) is actually a nice addition that the generic approach came up with on its own, but without the guided structure, the overall package is less polished and informative.

My own visualization (Figure 6) fills in what neither AI output captured well: a faceted view of every individual woman's trajectory. This is important because group means can mask a lot of variability. The faceted plot shows that while both groups generally trend in similar directions, there's substantial individual variation—some women in the null group actually gained calcium, while some in the exercise group lost it. These patterns would be invisible in either of the AI-generated group-means plots.

Taken together, these three visualizations show that planning your prompt really does matter.

The plan-informed approach gave me exactly what I needed, the generic prompt gave me a decent starting point but missed important details, and doing my own visualization let me explore aspects that neither AI approach thought to show.

4 Reflection

Working through this assignment brought together a lot of the skills we've been building over the semester in a way that felt more integrated than previous homework. The airports section was a good exercise in just the basics—building a tibble by hand, reshaping data with `pivot_wider()` and `pivot_longer()`, and creating a clear line chart. But writing the narrative was where I actually had to think carefully about what the data was saying. As I discussed regarding Figure 1, the post-pandemic recovery wasn't uniform at all, and without both the table and the chart side by side, I might have missed patterns like ATL's slight 2025 decline or PKX's weird 2022 dip due to zero-COVID.

The Monte Carlo section was probably the most conceptually interesting part. Writing the `generate_mcPoints()` function and running simulations at different sample sizes really drove home how Monte Carlo methods work—you can see the convergence happen right in front of you when you go from n = 10 to n = 10,000 in Figure 2. Using Quarto's multi-pane layout to display all four panels as a single figure was also a useful technical skill; it's a lot cleaner than trying to arrange plots manually and it handles the sub-captions automatically.

The GenAI comparison section was probably the most eye-opening. Going in, I figured that any decent prompt would get decent results from Claude. But the difference between my plan-informed prompt and the generic one was pretty stark. The planned version nailed the data tidying and produced exactly the visualization I wanted (Figure 3), while the generic version made reasonable but less useful choices—no error bars, generic group labels, and a less organized code structure. It really showed me that the more specific and structured your prompt is, the better the output, which is basically the same lesson as "garbage in, garbage out" but applied to AI tools.

Putting the whole thing together as a Quarto document also forced me to think about reproducibility in a practical way. Having all the code embedded in the document, with cross-references to specific figures and tables, means that anyone could re-render the entire analysis from scratch. That's a pretty powerful idea, and it's something I want to keep doing in future work.

Appendix A: GenAI Usage

This appendix documents all uses of generative AI tools in completing this assignment, in accordance with the course academic integrity policy.

Usage 1: Plan-Informed Calcium Visualization

- **Tool:** Anthropic Claude (claude-opus-4-6)
- **Date:** April 12, 2026
- **Purpose:** Generate R code to tidy the calcium dataset and create a group-means visualization, following my pre-written plan.
- **Prompt:** "I have a calcium.csv file from a longitudinal study of 31 women over 4 years measuring ulnar calcium levels. The data has 8 columns with duplicate names (y0,y1,y2,y3 repeated twice). The first 4 columns are the null treatment group (15 women, no intervention) and the last 4 are the diet/exercise group (16 women). Here is my plan: (1) Read the CSV preserving all 8 columns despite duplicate names. (2) Separate into two groups by column position. (3) Add subject IDs and group labels. (4) Rename time columns to Initial, Year1, Year2, Year3. (5) Pivot each group to long format. (6) Combine into one tidy data frame. (7) Remove NAs. (8) Create a line plot comparing mean calcium over time by group with standard error bars. Please write R code using tidyverse to execute this plan."
- **Response summary:** Claude generated a complete R script that followed each step of my plan. It used `read_csv()` to handle duplicate column names, `select()` by position to split groups, `set_names()` to relabel time columns, `pivot_longer()` and `bind_rows()` for reshaping, and `ggplot2` with `geom_errorbar()` for the final visualization. The code ran without errors and produced the plot shown in Figure 3.

Usage 2: Generic Prompt (Instructor-Provided)

- **Tool:** Anthropic Claude Sonnet 4.6 Extended
- **Date:** March 31, 2026
- **Purpose:** Instructor-provided baseline comparison for the GenAI prompting exercise.
- **Prompt:** Generic prompt provided by instructor (not authored by student).
- **Response:** The generic output produced two visualizations: a group means line plot (Figure 4) and an individual trajectories plot (Figure 5). These outputs were provided as-is by the instructor and are included for comparison purposes.

No other generative AI tools were used in the completion of this assignment. All other code, narrative text, and analysis were written by the student.

Appendix B: Code

All R code used in this document is shown below in a single block for reference.

```
# ============================================================
# SECTION 1: BUSIEST AIRPORTS ANALYSIS
# ============================================================

# --- Airport Data ---
airport_data <- tibble(
  airport = rep(c("ATL", "FRA", "PKX", "DXB", "LAX", "LHR"), each = 6),
  name = rep(c("Hartsfield-Jackson Atlanta", "Frankfurt Airport",
               "Beijing Daxing", "Dubai International",
               "Los Angeles International", "London Heathrow"), each = 6),
  year = rep(2020:2025, times = 6),
  passengers_M = c(
    42.9, 75.7, 93.7, 104.7, 108.1, 106.3,
    18.8, 24.8, 48.9, 59.4,  61.6,  63.2,
    16.0, 25.7, 17.5, 39.4,  52.0,  53.6,
    25.9, 29.1, 66.1, 86.9,  92.3,  95.2,
    28.8, 41.9, 65.9, 75.0,  76.6,  73.7,
    22.1, 19.4, 61.6, 79.2,  83.9,  84.5
  )
)

# --- Airport Table ---
airport_data |>
  select(airport, name, year, passengers_M) |>
  pivot_wider(names_from = year, values_from = passengers_M) |>
  rename(Airport = airport, Name = name) |>
  kable(digits = 1, align = c("l", "l", rep("r", 6))) |>
  kable_styling(latex_options = c("hold_position", "scale_down"),
                font_size = 10) |>
  column_spec(1, bold = TRUE)

# --- Airport Plot ---
airport_data |>
  filter(!is.na(passengers_M)) |>
  ggplot(aes(x = year, y = passengers_M, color = airport)) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 2.2) +
  scale_color_brewer(palette = "Set1") +
```

```r
  scale_x_continuous(breaks = 2020:2025) +
  scale_y_continuous(breaks = seq(0, 120, by = 30)) +
  labs(
    title = paste0("Annual Passenger Traffic at Six of the ",
                   "World's Busiest Airports"),
    subtitle = "2020\u20132025 | Figures in millions | Source: Wikipedia",
    x = "Year",
    y = "Passengers (Millions)",
    color = "Airport"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    plot.subtitle = element_text(size = 10, color = "gray40"),
    legend.position = "right"
  )

# ============================================================
# SECTION 2: MONTE CARLO NUMERICAL INTEGRATION
# ============================================================

# --- Helper Functions ---
generate_mcPoints <- function(n, x_min, x_max, y_min, y_max) {
  data.frame(
    x = runif(n, min = x_min, max = x_max),
    y = runif(n, min = y_min, max = y_max)
  )
}

make_mc_plot <- function(n, seed_val) {
  set.seed(seed_val)
  mc_data <- generate_mcPoints(n, 0, 20, 0, 0.16) |>
    mutate(flag = if_else(y > dchisq(x, df = 5), "above", "on/below"))

  rect_area <- 20 * 0.16
  estimate  <- round(rect_area * mean(mc_data$flag == "on/below"), 4)

  ggplot(mc_data, aes(x = x, y = y, color = flag)) +
    geom_point(alpha = 0.6, size = 1.2) +
    stat_function(fun = dchisq, args = list(df = 5), xlim = c(0, 20),
                  color = "darkblue", linewidth = 1) +
    scale_color_manual(values = c("above" = "#C8102E",
```

```r
                                  "on/below" = "#8DB600")) +
    labs(
      title = paste0("n = ", format(n, big.mark = ",")),
      subtitle = paste0("Estimate: ", estimate),
      x = "x", y = "y", color = NULL
    ) +
    theme_minimal(base_size = 10) +
    theme(legend.position = "bottom")
}

# --- Monte Carlo Plots (called inside Quarto layout div) ---
# fig-mc-n10:    make_mc_plot(10, seed_val = 42)
# fig-mc-n100:   make_mc_plot(100, seed_val = 99)
# fig-mc-n1000:  make_mc_plot(1000, seed_val = 7)
# fig-mc-n10000: make_mc_plot(10000, seed_val = 21)

# ============================================================
# SECTION 3: CALCIUM DATA - PLAN-INFORMED VISUALIZATION
# ============================================================

# --- Tidy the Calcium Data ---
calcium_raw <- read_csv("calcium.csv", show_col_types = FALSE)

null_group <- calcium_raw |>
  select(1:4) |>
  set_names(c("Initial", "Year1", "Year2", "Year3")) |>
  mutate(Subject = paste0("N", row_number()),
         Group = "Null Treatment") |>
  filter(!if_all(Initial:Year3, is.na))

exercise_group <- calcium_raw |>
  select(5:8) |>
  set_names(c("Initial", "Year1", "Year2", "Year3")) |>
  mutate(Subject = paste0("E", row_number()),
         Group = "Diet/Exercise")

null_long <- null_group |>
  pivot_longer(cols = Initial:Year3,
               names_to = "TimePoint",
               values_to = "Calcium")

exercise_long <- exercise_group |>
```

```r
  pivot_longer(cols = Initial:Year3,
               names_to = "TimePoint",
               values_to = "Calcium")

calcium_tidy <- bind_rows(null_long, exercise_long) |>
  mutate(TimePoint = factor(TimePoint,
                            levels = c("Initial", "Year1", "Year2", "Year3")))

calcium_tidy <- calcium_tidy |> drop_na(Calcium)

calcium_summary <- calcium_tidy |>
  group_by(Group, TimePoint) |>
  summarise(
    Mean = mean(Calcium),
    SD   = sd(Calcium),
    SE   = SD / sqrt(n()),
    .groups = "drop"
  )

# --- Plan-Informed Plot ---
ggplot(calcium_summary, aes(x = TimePoint, y = Mean,
                            color = Group, group = Group)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = Mean - SE, ymax = Mean + SE),
                width = 0.15, linewidth = 0.8) +
  scale_color_manual(values = c("Diet/Exercise" = "steelblue",
                                "Null Treatment" = "coral")) +
  labs(
    title = "Mean Ulnar Calcium Levels Over Time by Treatment Group",
    subtitle = "Error bars represent \u00b1 1 standard error",
    x = "Time Point",
    y = "Calcium Level (mg/cm\u00b2)",
    color = "Group"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 10, color = "gray40"),
    legend.position = "bottom"
  )
```

```r
# --- Generic Prompt Images (embedded via knitr::include_graphics) ---
# knitr::include_graphics("generic_group_means.png")
# knitr::include_graphics("generic_individual_lines.png")

# --- Student's Own Visualization ---
ggplot(calcium_tidy, aes(x = TimePoint, y = Calcium,
                         group = Subject, color = Group)) +
  geom_line(alpha = 0.45, linewidth = 0.6) +
  geom_point(alpha = 0.55, size = 1.5) +
  facet_wrap(~ Group) +
  scale_color_manual(values = c("Diet/Exercise" = "steelblue",
                                "Null Treatment" = "coral")) +
  labs(
    title = "Individual Calcium Trajectories by Treatment Group",
    x = "Time Point",
    y = "Calcium Level (mg/cm\u00b2)"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 13),
    legend.position = "none",
    axis.text.x = element_text(angle = 30, hjust = 1)
  )
```